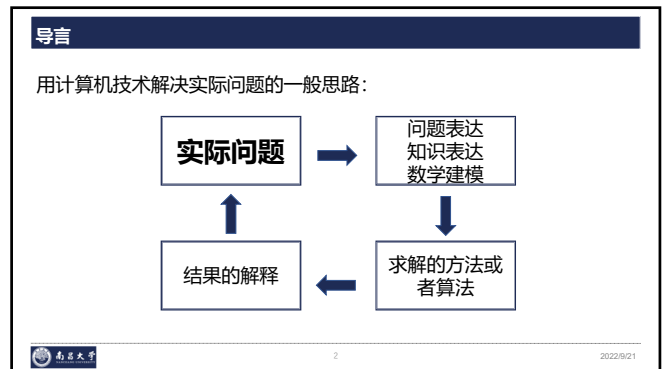
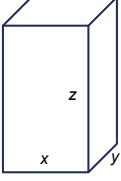




引言

例如：求侧面积为 150m^2 的体积最大的长方体？



设长宽高分别为 x, y, z
 侧面积为： $2(xy + yz + xz)$
 体积为： xyz

数学模型

$$\begin{aligned} \max \quad & xyz \\ \text{s.t.} \quad & 2(xy + yz + xz) = 150 \end{aligned}$$

3

引言

利用最优化技术中的算法，可以得到结果：

$$x = y = z = 5.0$$

结果的解释：长、宽、高都等于5米时，体积最大。

说明：在计算数学的课程中，主要关心求解的具体算法。

4

引言

在人工智能中，重点关注两个方面的内容：

① **问题的表示（知识的表示）**：即要找到问题的一种合适的表示方法。

我们将涉及到：

- 状态空间法
- 问题规约法
- 谓词逻辑法
- 语义网络法
- ...

5

引言

② **问题的求解**：从问题表示方法出发，找到一个合理的办法来求解。

常用的方法有：

- 搜索方法
- 推理方法
- 计算方法
- ...

6

本章主题

1. 状态空间法
2. 问题规约法
3. 谓词逻辑法
4. 语义网络法
5. ...



7

2022/9/21

本章主题

1. 状态空间法
2. 问题规约法
3. 谓词逻辑法
4. 语义网络法
5. ...

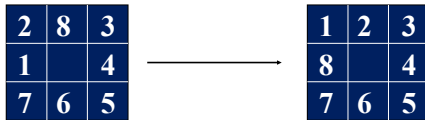


8

2022/9/21

状态空间法

- 在日常的一些智力游戏中（八数码、走八卦阵、走迷宫等），我们采用的策略：试着向前走，如果走不通，则往后退，不停地尝试，直到成功。



9

2022/9/21

状态空间法

- 类似地，在人工智能中，一种最基本的求解方法就是试探搜索法，即，通过在某个可能的解空间内寻找一个解来求解问题。
- 这种基于解答空间的问题表示和求解方法，即为**状态空间法**，它是以状态和算符为基础来表示和求解问题的。



10

2022/9/21

问题状态描述

□ 状态：

描述某一类不同事物间的差别而引入的一组最少变量 $q_0, q_1, \dots, q_n$ 的有序集合。

例：描述在坐的同学

变量可以有：年级、班级、姓名、性别、学号...

根据要解决的问题、从中选择最少的一组变量。

- ✓ 区分哪一个班：年级、班级
- ✓ 区分哪一位同学：姓名、性别、学号



11

2022/9/21

问题状态描述

矢量形式：

$$Q = [q_0, q_1, \dots, q_n]^T$$

其中，元素 $q_i (i = 0, 1, \dots, n)$ 为集合的分量，称作**状态变量**。

具体状态：给每一个状态变量一个具体的值（符号、数值等）。

矩阵形式：

$$Q = \begin{bmatrix} q_{11} & \dots & q_{1n} \\ q_{m1} & \dots & q_{mn} \end{bmatrix}$$



12

2022/9/21

问题状态描述

例：八数码问题

矢量形式的状态表示: $Q = [2, 8, 3, 1, 0, 4, 7, 6, 5]^T$

矩阵形式的状态表示: $Q = \begin{bmatrix} 2 & 8 & 3 \\ 1 & & 4 \\ 7 & 6 & 5 \end{bmatrix}$

2	8	3
1		4
7	6	5

问题状态描述

□ 算符 (操作符) :

使问题从一个状态变换到另一状态的手段。如: 走步、规则、数学算子、运算符号。

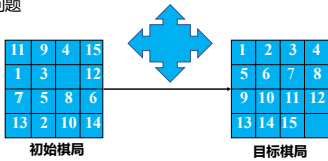
例: 描述在坐的同学

变量可以有: 年级、班级、姓名、性别、学号...

操作符: 入学、正常升级、毕业

问题状态描述

例: 十五数码问题



状态: 棋局

算符: 1、数字的上、下、左、右移动 (15×4=60个)

2、空格的上、下、左、右移动

问题状态描述

□ 问题的状态空间:

一个表示问题全部可能状态及其关系的图, 它包含了三个集合:

1、所有可能的问题初始状态集合S

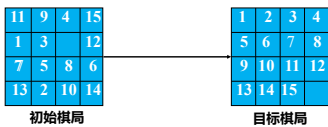
2、操作符集合F

3、目标状态集合G

因此, 可以把状态空间记作三元状态 (S, F, G)

问题状态描述

例: 十五数码问题



初始状态: 左图

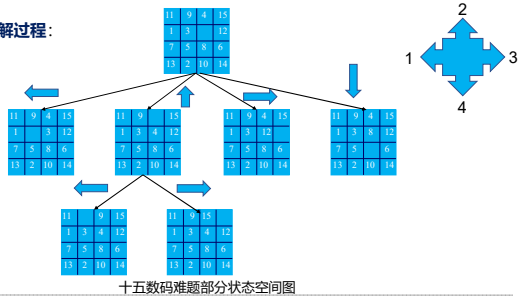
目标状态: 右图

操作符集合F = [空格左移、空格右移、空格上移、空格下移]



问题状态描述

可能的求解过程:



问题状态描述

□ 状态空间法：

从某一初始状态开始，每次施加一个操作符，递增地建立操作符序列，直到达到目标状态为止。

□ 状态空间法的问题：

寻找从初始状态到目标状态的某一个操作符序列。

□ 状态空间法的解：

从初始状态变换到目标状态的操作符序列。



状态空间法

基于解答空间的问题表示和求解方法，即为状态空间法，它是以状态和算符为基础来表示和求解问题的。

状态空间法三要点

(1) 状态

表示问题解法中每一步问题状况的数据结构；

(2) 算符

把问题从一种状态变换为另一种状态的手段；

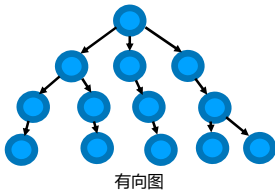
(3) 状态空间方法

基于解答空间的问题表示和求解方法，它是以状态和算符为基础来表示和求解问题的。

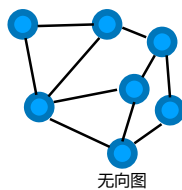
状态图示法

□ 状态空间法的表示方法：

用图来表示（借助于图论中某些技术）。图是由节点（不一定是有限的节点）的集合构成的。



有向图



无向图

状态图示法

父辈节点
或祖先

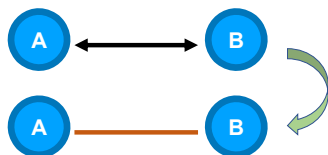


后继节点
或后裔

• 有向图

一对节点用弧线连接起来，从一个节点指向另一个节点，这种图叫做有向图。

状态图示法

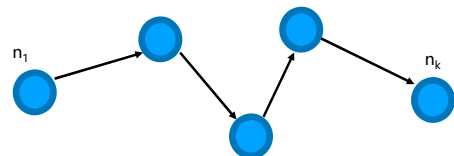


• 无向图

一对节点可能互为后裔，边用线段来表示。

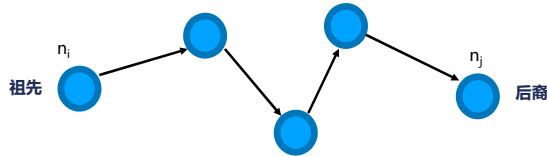
状态图示法

对于某一节点序列 $(n_1, n_2, \dots, n_j, \dots, n_k)$ ，当 $j = 2, 3, \dots, k$ 时，如果每一个节点 n_{j-1} 都有一个后继节点 n_j 存在，则将这一序列称为从节点 n_1 到 n_k 的长度为 k 的路径。



状态图示法

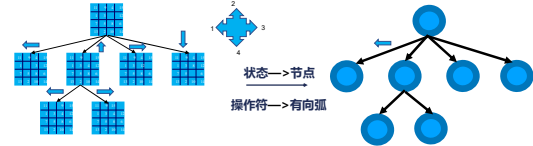
如果从节点 n_i 到 n_j 存在一条路径, 则称节点 n_j 是从节点 n_i 可到达的节点, 或者称 n_j 是 n_i 的后裔节点、 n_i 是 n_j 的祖先。



状态图示法

当用有向图来表示状态空间法时, 对应关系如下:

- ✓ 图中的一个节点对应于某一个状态。
- ✓ 途中的一个有向弧对应于某一个算符。
- ! 有向弧的旁边可以标以具体算符。



状态图示法

问题: 寻找从初始状态到目标状态的某个操作符序列。

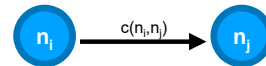


问题: 寻找图中初始节点 (对应初始状态) 到目标节点 (对应于目标状态) 的一条路径。

状态图示法

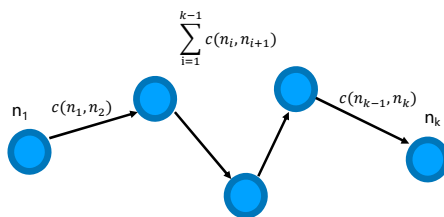
在某些情况下, 每个操作符作用、成本是不一样的, 需要引入代价的概念。

$c(n_i, n_j)$ 表示从节点 n_i 指向节点 n_j (相邻) 的那一段弧的代价。

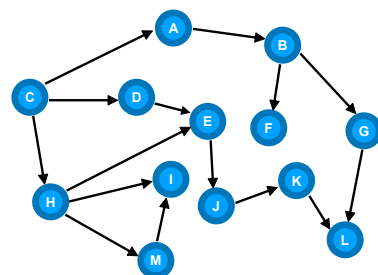


状态图示法

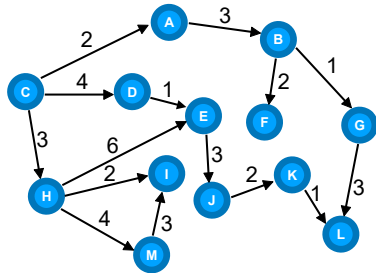
(不相邻的) 的两个节点间路径的代价等于连接该路径各个节点的所有弧线的代价之和:



状态图示法



状态图示法



状态图示法

引入代价的概念后，我们的问题可能是：

寻找初始节点到目标节点之间的代价最小的路径。

对应的原始问题：寻找从初始状态到目标状态的操作符代价之和最小的操作符序列。

状态图示法

利用图论的技术，我们要解决两个问题：

第一、找出初始节点到目标节点的一条路径。对应于寻找初始状态到目标状态的操作符序列。

第二、找到初始节点到目标节点的一条代价最小的路径。对应于寻找将初始状态变换到目标状态所用操作符代价之和最小的操作符序列。

状态图示法

□图的显式说明

对于显式说明，各节点及其具有代价的弧线由一张表明确给出。此表可能列出该图中的每一节点、它的后继节点以及连接弧线的代价。

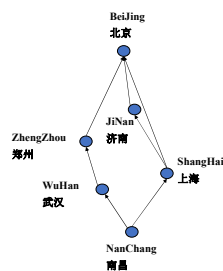
□图的隐式说明

节点的无限集合(s)作为起始节点是已知的。后继节点算符 r 也是已知的，它能作用于任一节点以产生该节点的全部后继节点和各连接弧线的代价。

状态空间表示举例

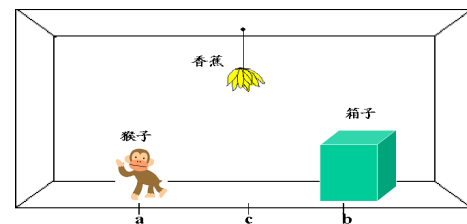
例1 路线规划

- Initial state
 - The journey starting from Nanchang
- Operators
 - Driving from a city to another city
- Goal state
 - Destination city — Beijing



状态空间表示举例

例2 猴子和香蕉问题



状态空间表示举例

解题过程:

- 用一个四元表列 (W, x, Y, z) 来表示这个问题状态.
 - W 猴子的水平位置
 - x 当猴子在箱子顶上时取 $x = 1$; 否则取 $x = 0$
 - Y 箱子的水平位置
 - z 当猴子摘到香蕉时取 $z = 1$; 否则取 $z = 0$

- 这个问题的操作 (算符) 如下:

- $\text{goto}(U)$: 表示猴子走到水平位置 U

$$(W, 0, Y, z) \xrightarrow{\text{goto}(U)} (U, 0, Y, z)$$



37

2022/9/21

状态空间表示举例

- $\text{pushbox}(V)$: 表示猴子把箱子推到水平位置 V

$$(W, 0, W, z) \xrightarrow{\text{pushbox}(V)} (V, 0, V, z)$$

- 注意: 要应用算符 $\text{pushbox}(V)$, 就要求规则的左边, 猴子与箱子必须在同一位置上, 并且, 猴子不是箱子顶上。这种强加于操作的适用性条件, 叫做产生式规则的**先决条件**。

- climbbox : 猴子爬上箱顶

$$(W, 0, W, z) \xrightarrow{\text{climbbox}} (W, 1, W, z)$$

应用算符 climbbox 的先决条件是什么?



38

2022/9/21

状态空间表示举例

- grasp : 表示猴子摘到香蕉

$$(c, 1, c, z) \xrightarrow{\text{grasp}} (c, 1, c, 1)$$

令初始状态为 $(a, 0, b, 0)$ 。这时, $\text{goto}(U)$ 是唯一适用的操作, 并导致下一状态 $(U, 0, b, 0)$ 。现在有3个适用的操作, 即 $\text{goto}(U)$, $\text{pushbox}(V)$ 和 climbbox (若 $U=b$)。把所有适用的操作继续应用于每个状态, 就能够得到状态空间图。把该初始状态变换为目标状态的操作序列为:

$\{\text{goto}(b), \text{pushbox}(c), \text{climbbox}, \text{grasp}\}$

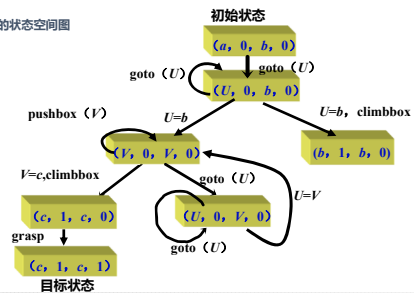


39

2022/9/21

状态空间表示举例

猴子和香蕉问题的状态空间图

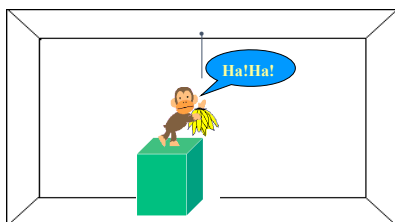


40

2022/9/21

状态空间表示举例

猴子和香蕉问题自动演示:



41

2022/9/21

状态空间表示举例

例3 传教士和野人问题 (Missionaries & Cannibals Problem, MC问题)



42

2022/9/21

状态空间表示举例

- 状态：问题在某一时刻所处的“位置”，“情况”等
- 根据问题所关心的因素，一般用向量形式表示，每一位表示一个因素。

状态可有多种表示方法：
(左岸传教士数, 右岸传教士数, 左岸野人数, 右岸野人数, 船的位置)
或简化表示为：
(左岸传教士数, 左岸野人数, 船的位置)

初始状态: (0, 0, 0) 0: 右岸
目标状态: (3, 3, 1) 1: 左岸



43

2022/9/21

状态空间表示举例

□ 算子 (算符, 操作符 Operator) —— 使状态发生改变的操作

□ MC问题中的算子

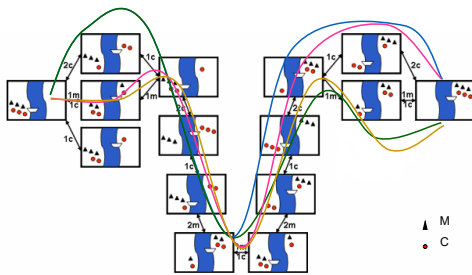
- 将传教士或野人运到河对岸
- Move-1m1c-lr: 将一个传教士(m)和一个野人(c)从左岸(L)运到右岸(R)
- 所有可能操作
 - Move-1m1c-lr
 - Move-1m1c-rl
 - Move-2c-lr
 - Move-2c-rl
 - Move-2m-lr
 - Move-2m-rl
 - Move-1c-lr
 - Move-1c-rl
 - Move-1m-lr
 - Move-1m-rl



44

2022/9/21

状态空间表示举例



45

2022/9/21

本章主题

1. 状态空间法
2. 问题规约法
3. 谓词逻辑法
4. 语义网络法
5. ...



46

2022/9/21

问题规约法

• 例:
求积分 $\int (e^x + \cos x + x^2) dx$

✓ 解法1:

$$\begin{aligned} \int (e^x + \cos x + x^2) dx \\ &= \int e^x dx + \int \cos x dx + \int x^2 dx \\ &= e^x + \sin x + \frac{x^3}{3} \end{aligned}$$

✓ 解法2:

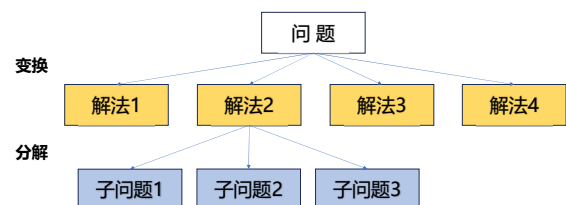
✓ 解法3:



47

2022/9/21

问题规约法



48

2022/9/21

问题状态描述

□ 问题规约法：

从已知问题的描述出发，通过一系列**变换或分解**将问题最终变为一个子问题集合，这些子问题的解可以直接得到，从而解决初始问题。

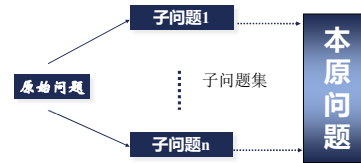
问题规约法由三个部分组成：

- ✓ 一个初始问题描述
- ✓ 一套将问题变换或分解为子问题的操作符
- ✓ 一套本原问题（解可以直接得到的简单问题）描述

问题规约法

□ 问题规约法思想：

先把问题分解为子问题及子-子问题，然后解决较小的问题。对该问题的某个具体子集的解答就意味着对原始问题的一个解答



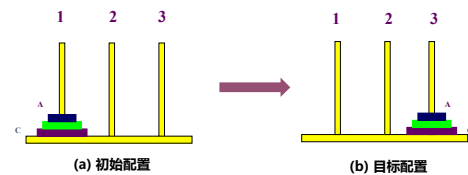
问题规约法

□ 问题归约的实质：

- 从目标(要解决的问题)出发**逆向推理**，建立子问题以及子问题的子问题，直至最后把初始问题归约为一个平凡的本原问题集合。

梵塔问题

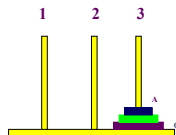
□ 梵塔问题（三圆盘）



梵塔问题

□ 约束条件

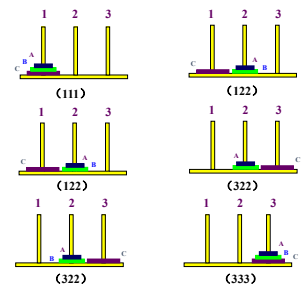
- (1) 一次只能移动一个圆盘；
- (2) 只有柱子最上面的圆盘可以移动；
- (3) 大圆盘不能放在小圆盘上面。



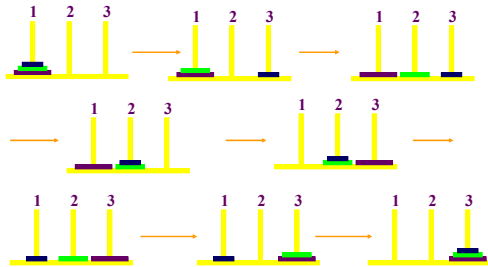
梵塔问题

□ 将原始问题归约为一个较简单问题集合

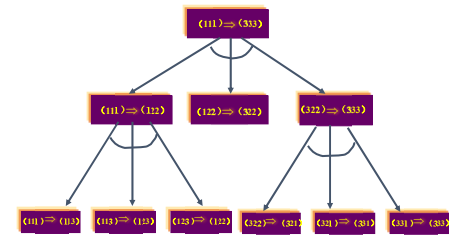
- (1) 移动圆盘A和B至柱子2的双圆盘难题；
- (2) 移动圆盘C至柱子3的单圆盘难题；
- (3) 大圆盘不能放在小圆盘上面。



梵塔问题



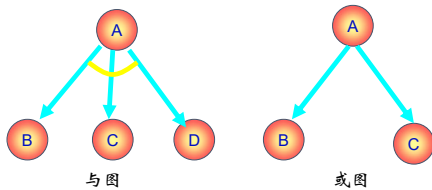
梵塔问题归约图 (与或图)



与或图表示

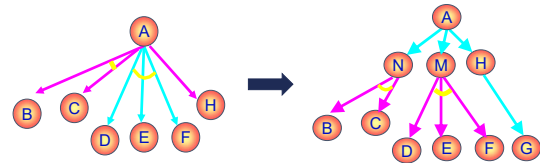
与图、或图、与或图

一般, 用一个似图结构来表示把问题归约为后继问题的替换集合, 这一似图结构叫做问题归约图(与或图)。



与或图表示

通常把某些附加节点引入此结构, 以便使含有一个以上后继问题的集合能够聚集在它们各自的父节点之下。



与或图表示

父节点、子(后继)节点、弧线

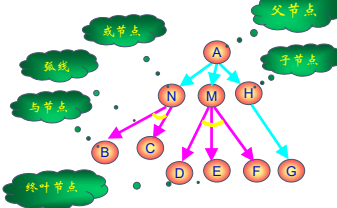
起始节点: 对于原问题描述的节点

终叶节点: 对应于本原问题的节点

或节点: 只要解决某个问题就可解决其父问题的节点集合, 如 (M, N, H)。

与节点: 只有解决所有子问题, 才能解决其父问题的节点集合, 如 (B, C) 和 (D, E, F)。各个节点之间用一段小圆弧连接标记。

与或图: 由与节点及或节点组成的结构图。



与或图表示

可解节点的一般定义

终叶节点是可解节点(因为它们与本原问题相关联)。

如果某个非终叶节点含有或后继节点, 那么只要有一个后继节点是可解的时, 此非终叶节点就是可解的。

如果某个非终叶节点含有与后继节点, 那么只有其全部后继节点为可解时, 此非终叶节点才是可解的。

与或图表示

□ 不可解节点的一般定义

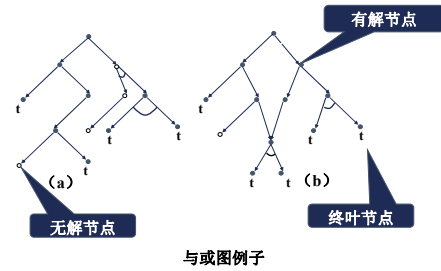
- 没有后裔的非终叶节点为不可解节点。
- 如果某个非终叶节点含有或后继节点，那么只有当其全部后裔为不可解时，此非终叶节点才是不可解的。
- 如果某个非终叶节点含有与后继节点，那么只要当其后裔有一个为不可解时，此非终叶节点就是不可解的。



61

2022/9/21

与或图表示



62

2022/9/21

与或图表示

□ 与或图构成规则

- 与或图中的每个节点代表一个要解决的单一问题或问题集合。起始节点对应于原始问题。
- 对应于本原问题的节点，叫做终叶节点。
- 对于把算符应用于问题A的每种可能情况，都把问题变换为一个子问题集合；有向弧线自A指向后继节点，表示所求得子问题集合，这些子问题节点叫做或节点。
- 一般对于代表两个或两个以上子问题集合的每个节点，有向弧线从此节点指向此子问题集合中的各个节点，这些子问题节点叫做与节点。



63

2022/9/21